

Software Requirements Specification (SRS)

Project Name: Project Board

1. Introduction

1.1 Purpose

The purpose of this document is to outline the Software Requirements Specification (SRS) for the **Project Board** application. It describes the system's behavior, the different roles, the modules it contains, and the expected features.

1.2 Scope

Project Board is a comprehensive project management system designed specifically for academic environments. It allows faculty members to seamlessly manage, track, and evaluate student projects. Simultaneously, it provides students with a collaborative platform to create project groups, select mentors, submit projects, and track their academic progress.

1.3 Definitions and Acronyms

- **SRS:** Software Requirements Specification
 - **UI:** User Interface
 - **DB:** Database
 - **Admin:** System Administrator
-

2. Overall Description

2.1 Product Perspective

Project Board is a web-based application built using ASP.NET (C#) with an SQL Server database. It serves as a centralized hub for project-related activities within an educational institution, eliminating manual submissions and tracking.

2.2 User Classes and Characteristics

1. **Student:** Can form groups, search for mentors, propose project ideas, submit progress reports, and view faculty feedback.
2. **Faculty / Mentor:** Can review student proposals, accept or reject group mentorship requests, track project milestones, and provide grades/feedback.
3. **Admin:** Has full access to the system. Can manage user accounts (Students and Faculty), oversee all ongoing projects, and ensure the system runs smoothly.

2.3 Operating Environment

- **Server-side:** ASP.NET framework, IIS Server.
 - **Database:** Microsoft SQL Server.
 - **Client-side:** Modern web browsers (Chrome, Firefox, Safari, Edge) with JavaScript enabled.
-

3. System Features

3.1 Login and Signup Module

- Secure registration and authentication for Students, Faculty, and Admins.
- Role-based redirection upon successful login.
- Password encryption and session management.

3.2 Group Creation and Management Module

- Students can create a new project group and invite other students.
- Students can join existing groups using group codes or invites.
- Group leaders have the authority to manage group members.

3.3 Mentor Selection Module

- Groups can browse available faculty members and send mentorship requests.
- Faculty can view pending requests and choose to accept or decline based on availability and project topics.

3.4 Project Submission and Tracking Module

- Students can submit project proposals, documentations, and final code repositories.
- Track status of submissions (Pending, Accepted, Requires Changes, Rejected).
- Maintain a timeline or task list for project milestones.

3.5 Project Evaluation and Feedback Module

- Faculty can grade and evaluate the submitted work.
- Faculty can leave comments, suggestions, and request modifications on project deliverables.

3.6 User, Faculty, and Admin Management Modules

- **User Management:** Students can update their profile information.
 - **Faculty Management:** Faculty can update their domains of expertise.
 - **Admin Management:** Admin can add, update, or remove users and manage global system settings.
-

4. Database Schema

4.1 Core Entities

- **Users:** Stores user information including roles (Student, Faculty, Admin), authentication details, and enrollment numbers.
- **Technologies:** Master list of technologies for projects and faculty expertise.
- **Faculty:** Maps faculty members to their areas of technological expertise.

4.2 Group and Project Entities

- **Groups:** Manages student project groups, their leaders, mentors, and current status.
- **GroupMembers:** Tracks students belonging to groups and their join status.
- **Projects:** Stores project proposals, titles, functionality descriptions, and their approval status.
- **ProjectKeywords:** Stores tags and keywords associated with projects for easier categorization.
- **GroupMentorRejections:** Tracks rejected mentorship requests to avoid duplicate requests.

4.3 Task and Evaluation Entities

- **Task:** Manages hierarchical tasks assigned between mentors and leaders, or leaders and members. Tracks due dates, submissions, and feedback.
 - **Appeals:** Manages student requests for task review or changes, linking to specific tasks and reviewers.
 - **RejectionLogs:** Maintains an audit trail for rejections across groups, projects, tasks, and appeals.
 - **Notifications:** Stores system alerts and messages for users.
-

5. Non-Functional Requirements

5.1 Performance Requirements

- The system shall handle multiple concurrent users without significant degradation in response time.
- Page load times should not exceed 3 seconds under normal network conditions.

5.2 Security Requirements

- Passwords must be hashed before storing in the database.
- Authorization checks must be implemented on every page to prevent unauthorized access.
- Protection against common web vulnerabilities (e.g., SQL Injection, XSS).

5.3 Reliability and Availability

- The system should ensure 99% uptime during academic semesters.
- Regular database backups should be maintained.

5.4 Usability

- The user interface must be intuitive, modern, and mobile-responsive.
 - Clear error messages should be displayed for invalid actions.
-

6. Future Enhancements

- Integration with external repositories (e.g., GitHub, GitLab).
 - Email or SMS notifications for important project updates.
 - Advanced analytics and reporting dashboards for the Admin.
-

Generated by Antigravity